

Análisis de un Perceptrón Multicapa (MLP) para Reconocimiento de Patrones

Baggio Lucas, Cordoba Rodrigo, Gauna Lesteyme Lucas, Gonzalez Urbieta Enzo, Lezcano Rafael

Grupo: Tortugas V2

Inteligencia Artificial - 5to Nivel ISI - Ciclo 2025

Universidad Tecnológica Nacional, Resistencia (Chaco), Argentina

(LucasBaggio2000, CordobaRodrigo, Lks.Gauna, EnzoGonzalez1405
RafaLezcano72,)@gmail.com

Resumen

El presente trabajo práctico detalla la implementación y evaluación de un Perceptrón Multicapa (MLP) desde cero, utilizando el algoritmo de retropropagación (backpropagation) con momento. El objetivo principal es analizar el rendimiento de la red para un problema de clasificación de patrones, reconociendo las letras “b”, “d”, y “f” en una cuadrícula de 10x10. Se evalúa la precisión del modelo, medida por el Error Cuadrático Medio (MSE), bajo 144 configuraciones distintas, variando la arquitectura de la red como son la cantidad de capas y de neuronas, las funciones de activación, los hiperparámetros de entrenamiento, como la tasa de aprendizaje y momento, el tamaño del conjunto de datos y la proporción de división entre entrenamiento y validación. Finalmente, se presenta una interfaz de usuario interactiva capaz de clasificar patrones distorsionados en tiempo real.

Palabras Clave: Perceptrón Multicapa, MLP, Backpropagation, Momento, Reconocimiento de Patrones, Error Cuadrático Medio, Hiperparámetros, Redes Neuronales.

1. Introducción

El Perceptrón Multicapa (MLP) es una arquitectura fundamental de red neuronal artificial de tipo feedforward en el ámbito del aprendizaje supervisado. A diferencia de su predecesor, el perceptrón simple, el MLP es capaz de aprender relaciones no lineales complejas mediante la inclusión de una o más capas ocultas entre la capa de entrada y la de salida.

El objetivo de este estudio, enmarcado en la consigna del Trabajo Práctico Integrador, es doble. En primer lugar, se busca implementar un algoritmo MLP funcional desde cero para comprender en profundidad sus mecanismos internos, tales como la propagación hacia adelante (feedforward) y la retropropagación del error (backpropagation) con la optimización de momento. En segundo lugar, se utiliza esta implementación para realizar un análisis exhaustivo de su rendimiento en una tarea de reconocimiento de patrones.

El problema abordado consiste en la clasificación de patrones de 10x10 píxeles (100 entradas) que representan las letras “b”, “d” y “f”. El análisis se centra en evaluar cómo la precisión de la red, medida por el Error Cuadrático Medio (MSE), se ve afectada por distintas configuraciones. Estas configuraciones exploran el impacto de la arquitectura (una o dos capas ocultas), la función de activación (Lineal o Sigmoide), los hiperparámetros de entrenamiento (tasa de aprendizaje y momento), el tamaño del conjunto de datos y la proporción de la división entrenamiento/validación.

Este informe presenta el marco teórico del MLP, la metodología de simulación empleada, un análisis detallado de los resultados obtenidos y las conclusiones sobre las configuraciones óptimas para este problema.

2. Marco Teórico

2.1 Perceptrón Multicapa (MLP)

Un MLP es una red neuronal feedforward que consta de una capa de entrada, una o más capas ocultas y una capa de salida. Cada capa está compuesta por neuronas (o nodos), y cada neurona de una capa está completamente conectada a las neuronas de la capa siguiente.

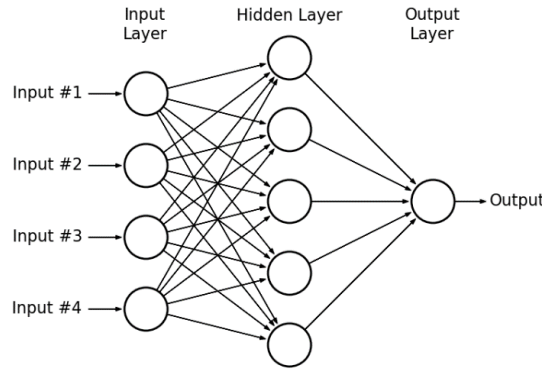


Figura 1: Representación gráfica de un Perceptrón Multicapa

- **Capa de Entrada:** Recibe los datos brutos. En este trabajo, consta de 100 neuronas, una por cada píxel del patrón 10x10.
- **Capas Ocultas:** Son las responsables de extraer características y aprender representaciones internas de los datos. Sus neuronas aplican una función de activación no lineal.
- **Capa de Salida:** Produce el resultado final. Para este problema de clasificación de 3 letras, la capa de salida tiene 3 neuronas, cada una correspondiendo a la probabilidad de pertenencia a una de las clases ("b", "d", "f") mediante codificación one-hot.

2.2 Propagación Hacia Adelante (Feedforward)

Es el proceso en el que la red calcula una salida. Para una neurona j en una capa l , la entrada neta net_j se calcula como la suma ponderada de las salidas o_i de la capa anterior $l - 1$, más un sesgo (bias) b_j :

$$net_j = \sum_i (w_{ij} * o_i) + b_j$$

La salida o_j de esta neurona es el resultado de aplicar una función de activación f a su entrada neta:

$$o_j = f(net_j)$$

Este proceso se repite desde la capa de entrada hasta la capa de salida, que produce la predicción final de la red.

2.3 Retropropagación del Error (Backpropagation)

El entrenamiento se realiza mediante backpropagation, un algoritmo que ajusta los pesos (w_{ij}) y sesgos (b_j) de la red para minimizar una función de pérdida. En este caso se utiliza el Error Cuadrático Medio (MSE) el cual se define como:

$$\text{MSE} = \frac{1}{N} \sum_{p=1}^N \frac{1}{M} \sum_{k=1}^M (t_k^p - o_k^p)^2$$

Donde N es el número de muestras, M es el número de neuronas de salida, t_k^p es el valor objetivo (target) y o_k^p es la salida de la red (output) para la muestra p y la neurona k .

El algoritmo calcula el gradiente del error con respecto a cada peso y sesgo, propagando el error desde la capa de salida hacia atrás.

2.4 Optimización con Momento

Para acelerar la convergencia y evitar mínimos locales, la implementación utiliza un término de **Momento**. En lugar de actualizar el peso solo con el gradiente actual, se añade una fracción (μ) de la actualización de peso anterior ($\Delta w_{ij}(t-1)$).

$$\Delta w_{ij}(t) = (\eta * \delta_j * o_i) + (\mu * \Delta w_{ij}(t-1))$$

Donde η es la tasa de aprendizaje y δ_j es el término de error de la neurona j . Un valor de momento alto (p. ej. 0.9) permite al algoritmo mantener la inercia de su dirección anterior, facilitando la superación de pequeños mínimos locales.

3. Descripción de la Solución Implementada

3.1 Generación de Datos

El problema se centra en el reconocimiento de las letras “b”, “d” y “f” en una cuadrícula de 10x10. Para este fin, se generaron tres conjuntos de datos principales con 100, 500 y 1000 muestras cada uno. Cada conjunto se compuso de un 10% de patrones puros (sin distorsión) y un 90% de patrones distorsionados. La distorsión se introdujo alterando aleatoriamente un porcentaje de píxeles del patrón puro original.

Para el análisis de hiperparámetros, estos conjuntos de datos fueron a su vez subdivididos en particiones de entrenamiento y validación, empleando tres proporciones distintas: 90%/10%, 80%/20% y 70%/30%.

3.2 Implementación del MLP

Se desarrolló una clase MLP en Python que encapsula la lógica de la red. La inicialización de la clase requiere la definición de la arquitectura (p. ej., [100, 10, 3]), junto con una lista de las funciones de activación y sus derivadas correspondientes para cada capa. Los pesos sinápticos se inicializan con valores aleatorios, mientras que los sesgos se establecen en cero.

El método de entrenamiento (**fit**) implementa el bucle principal de épocas. Dentro de cada época, se aplica la retropropagación con momento para cada muestra del conjunto de entrenamiento. Al finalizar la época, se calcula el MSE tanto para los datos de entrenamiento como para los de validación. Se incorporó una estrategia de parada temprana (early stopping) con una paciencia de 20 épocas, que detiene el entrenamiento si el error de validación no presenta mejoras. Adicionalmente, el método incluye manejo de excepciones para la divergencia numérica producto de errores de tipo `inf` o `nan`.

Finalmente, el método de predicción (**predict**) se optimizó para operar en modo de batch (por lotes), permitiendo una evaluación significativamente más rápida del conjunto de validación en comparación con la predicción muestra por muestra.

3.3 Entorno de Simulación

Para la evaluación sistemática de la red, se implementó un script de simulación que itera sobre un espacio de búsqueda de 144 combinaciones. Este espacio se define por el producto cartesiano de los siguientes hiperparámetros y configuraciones:

- **Tamaño de Dataset:** {100, 500, 1000}
- **Proporción de Split:** {90/10, 80/20, 70/30}
- **Arquitecturas:** {[100, 10, 3], [100, 10, 5, 3]}
- **Tasa de Aprendizaje:** {0.01, 0.1}
- **Momento:** {0.5, 0.9}
- **Activación:** {Sigmoide, Lineal}

Cada una de las 144 simulaciones se ejecutó durante un máximo de 20 épocas, registrando el MSE final de validación. Los resultados fueron almacenados en un archivo CSV para su posterior análisis.

3.4 Interfaz de Usuario

Se implementó una interfaz de usuario interactiva (ver Figura 2) utilizando anywidget dentro del entorno del notebook. Dicha interfaz presenta una cuadrícula de 10x10 que permite al usuario dibujar un patrón. Incluye controles para cargar los patrones puros ("b", "d", "f"), un deslizador para aplicar distorsión aleatoria, y un botón de predicción. Al invocar la predicción, la interfaz extrae el estado actual de la cuadrícula (un vector de 100 entradas), lo suministra al modelo MLP previamente entrenado y despliega la clase resultante junto con el vector de confianza, es decir, la salida de las 3 neuronas finales.

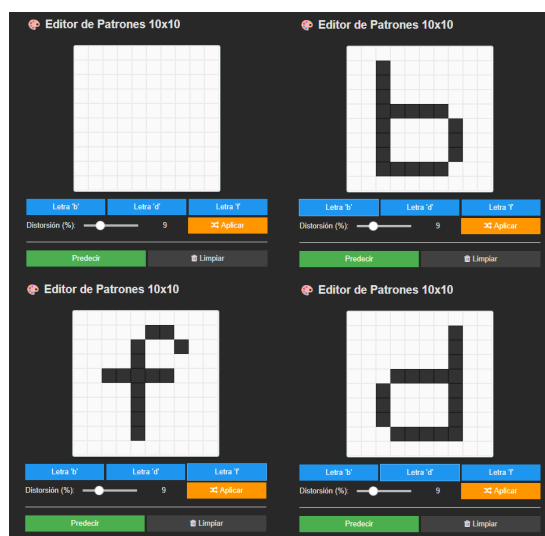


Figura 2: Interfaz gráfica interactiva implementada para el dibujo y clasificación de patrones 10x10.

4. Análisis de Resultados de Simulación

El análisis se efectuó sobre los resultados de las 144 simulaciones. De este conjunto, 107 experimentos (74.3%) completaron el entrenamiento exitosamente, mientras que 37 (25.7%) fallaron al presentar divergencia numérica. El análisis se efectuó sobre los resultados de las 144 simulaciones. De este conjunto, 107 experimentos (74.3%) completaron el entrenamiento exitosamente, mientras que 37 (25.7%) fallaron al presentar divergencia numérica.

4.1 Funciones de Activación: Sigmoide vs. Lineal

La elección de la función de activación en las capas ocultas fue el factor más determinante del éxito. La función Lineal se mostró extremadamente inestable: la totalidad

de las 37 simulaciones que divergieron emplearon esta función, un resultado observado exclusivamente en combinación con la tasa de aprendizaje alta (0.1).

Por el contrario, las 72 simulaciones que utilizaron la función Sigmoide convergieron exitosamente, exhibiendo un rendimiento estable y un Error Cuadrático Medio consistentemente bajo, como se ilustra en la Figura 3.

Este resultado es coherente con la naturaleza del problema. La función Lineal, al no ser acotada, es susceptible de provocar una explosión del gradiente (gradient explosion) durante la retropropagación, un efecto que se magnifica con tasas de aprendizaje elevadas. La función Sigmoide, al acotar las salidas de las neuronas al rango $[0, 1]$, mantiene la estabilidad del gradiente y es intrínsecamente más adecuada para este problema de clasificación que busca salidas probabilísticas.

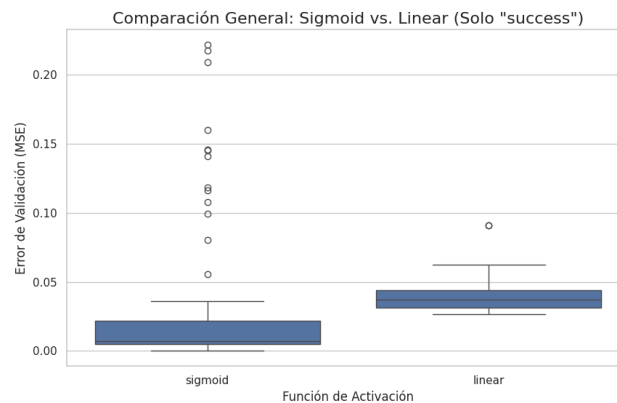


Figura 3: Comparativa del MSE de Validación entre funciones de activación. La función Lineal muestra alta variabilidad y divergencia, mientras que Sigmoide es estable.

4.2 Arquitectura: 1 vs. 2 Capas Ocultas

Para evaluar el impacto de la profundidad de la red, se compararon las arquitecturas filtrando solo los resultados de las simulaciones estables (con función Sigmoide). Las arquitecturas fueron una capa oculta de 10 neuronas ([100, 10, 31]) y dos capas ocultas de 10 y 5 neuronas ([100, 10, 5, 31]).

Como se observa en la Figura 4, la arquitectura de dos capas ocultas obtuvo un MSE de validación promedio y mediano inferior al de la arquitectura de una sola capa. Esto sugiere que la profundidad adicional, incluso con un número reducido de neuronas en la segunda capa, permite a la red capturar de manera más efectiva la complejidad y las características no lineales de los patrones de entrada.

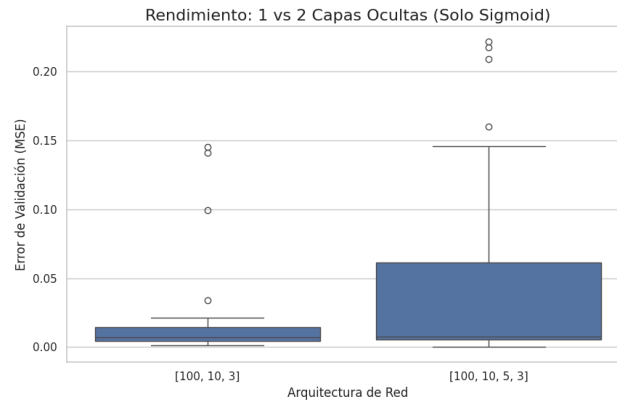


Figura 4: Comparativa del MSE de Validación entre arquitecturas de 1 y 2 capas ocultas (solo Sigmoide).

4.3 Hiperparámetros: Tasa de Aprendizaje y Momento

Dentro del subconjunto de simulaciones con Sigmoide, los hiperparámetros de entrenamiento tuvieron un impacto claro. Se observó que una tasa de aprendizaje (LR) de 0.1 resultó consistentemente mejor, produciendo un MSE más bajo que una tasa de 0.01 (Figura 5). Esto indica que, para el número limitado de épocas (20) de la simulación, un LR de 0.01 era demasiado conservador, resultando en una convergencia subóptima.

En cuanto al momento (Momentum), un valor alto de 0.9 superó de manera significativa al valor de 0.5 (Figura 6). Este hallazgo es esperable, ya que un momento elevado permite al algoritmo de descenso de gradiente acumular inercia, acelerar en direcciones consistentes y superar mínimos locales, convergiendo más rápido a una solución de calidad.

Por lo tanto, la combinación de LR=0.1 y Momento=0.9 se identificó como la más efectiva para este problema.

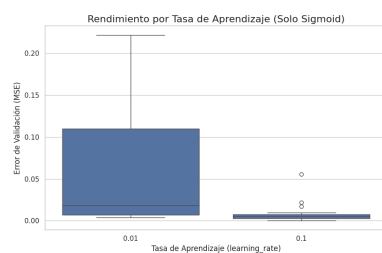


Figura 5: Impacto de la Tasa de Aprendizaje.

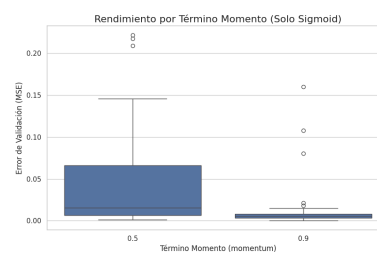


Figura 6: Impacto del Término Momento.

4.4 Tamaño del Conjunto de Datos

El análisis del tamaño del dataset (Figura 7) muestra una tendencia clara: a medida que el tamaño del conjunto de datos aumenta (de 100 a 500 y a 1000), el error de validación promedio disminuye. Esto demuestra que la red generaliza mejor y se vuelve más robusta cuando se la entrena con más ejemplos.

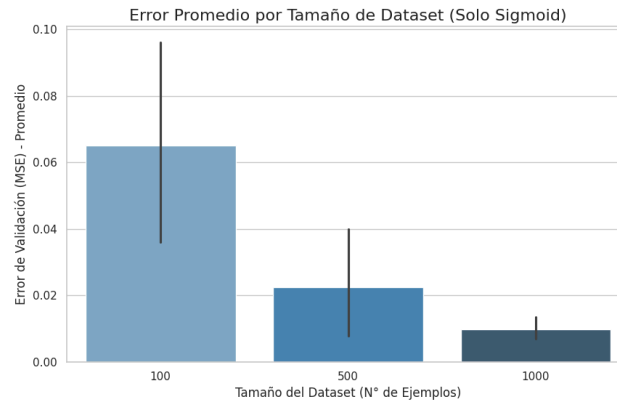


Figura 7: Error de Validación promedio según el tamaño del dataset (solo Sigmoide).
A más datos, menor error.

4.5 Modelo Óptimo y Matriz de Confusión

El análisis agregado de las 144 simulaciones (Figura 8) confirmó que las 10 mejores configuraciones, en términos de menor MSE de validación, compartieron un núcleo común de hiperparámetros: **Activación Sigmoide, LR=0.1 y Momento=0.9**.

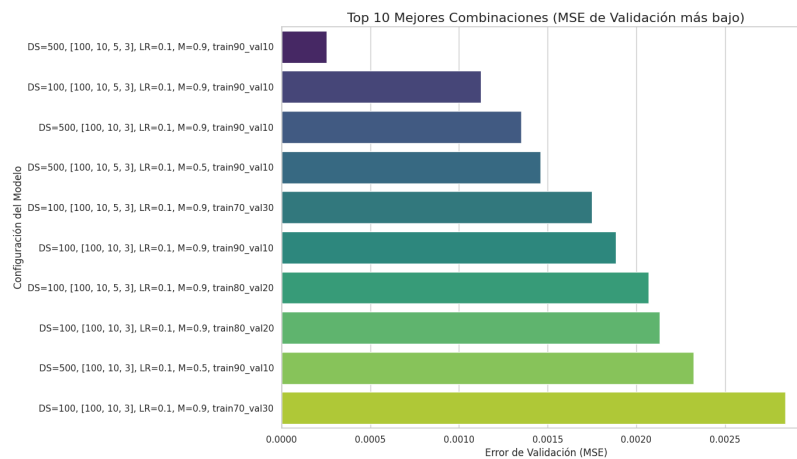


Figura 8: Top 10 mejores configuraciones de hiperparámetros, ordenadas por MSE de Validación.

Basado en esto, se seleccionó un modelo con parámetros casi óptimos (Dataset=500, Split=80/20, 2 capas, Sigmoide, LR=0.1, M=0.9) y se re-entrenó durante 300 épocas para asegurar una convergencia completa. La curva de aprendizaje resultante (Figura 9) muestra una convergencia rápida y estable, alcanzando un MSE de validación final de 0.00579.

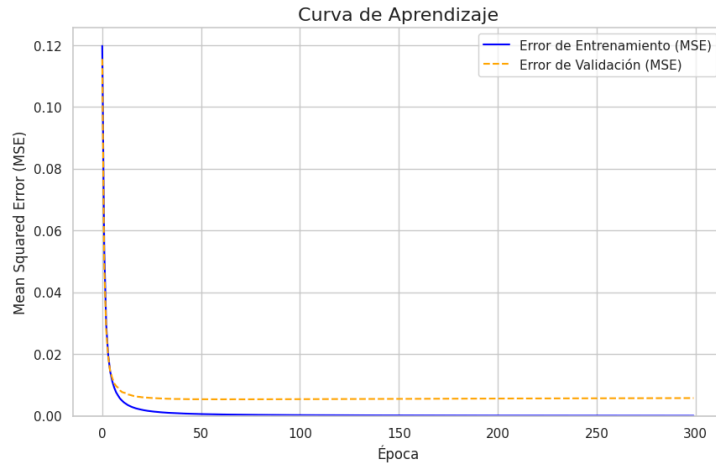


Figura 9: Curva de aprendizaje del modelo optimizado (300 épocas). Se observa una convergencia rápida y estable, con un MSE de validación final de 0.00579.

Al evaluar este modelo final sobre el conjunto de validación, se obtuvo una precisión del 98%. La matriz de confusión (Tabla 1) detalla un rendimiento casi perfecto, observándose únicamente dos errores de clasificación (dos instancias de “f” clasificadas erróneamente como “d”).

Predicción	b	d	f
Real b	31	0	0
Real d	0	33	0
Real f	0	2	34

Tabla 1: Matriz de confusion

5. Conclusión

El modelo final optimizado alcanzó una precisión del 98% en la validación, demostrando la alta capacidad del MLP implementado para resolver el problema de reconocimiento de patrones propuesto. La interfaz de usuario desarrollada validó además la capacidad del modelo para clasificar patrones nuevos y distorsionados en tiempo real.

En este trabajo se implementó exitosamente un Perceptrón Multicapa con retropropagación y momento, y se validó su capacidad para resolver un problema de clasificación de patrones 10x10. El análisis exhaustivo de 144 configuraciones experimentales permitió extraer conclusiones robustas sobre la parametrización de la red.

Se demostró que la función de activación Sigmoide es un componente esencial para la estabilidad del entrenamiento, en contraste con la función Lineal, que resultó inestable y propensa a la divergencia. Se determinó que una arquitectura más profunda (dos capas ocultas) ofrece una mejor capacidad de generalización que una más superficial (una capa) para capturar la complejidad de las características de los patrones.

Además, el análisis de hiperparámetros reveló que una configuración «agresiva», combinando una tasa de aprendizaje de 0.1 y un momento de 0.9, fue la más eficiente para una convergencia rápida. Finalmente, se confirmó la tendencia esperada de que un mayor volumen de datos (1000 muestras) reduce el error de validación, mejorando la robustez del modelo.

La sinergia de estos hallazgos permitió entrenar un modelo final optimizado que alcanzó una precisión del 98% en el conjunto de validación. La interfaz de usuario

desarrollada validó la aplicación práctica del modelo, clasificando con éxito patrones nuevos y distorsionados en tiempo real, demostrando la alta capacidad del MLP implementado y cumpliendo con todos los objetivos propuestos.

Referencias

1. Popescu, Marius-Constantin & Balas, Valentina & Perescu-Popescu, Liliana & Mastorakis, Nikos. (2009). Multilayer perceptron and neural networks. WSEAS Transactions on Circuits and Systems. 8.
2. Hilera, José & Martinez Hernando, Victor. (1995). Redes neuronales artificiales : fundamentos, modelos y aplicaciones / José Ramón Hilera González, Victor José Martínez Hernando.
3. Delashmit, W. (2005). Recent Developments in Multilayer Perceptron Neural Networks.